

Foundations for Programming Languages

3:

Compilation of statements

Statements in C

Assertion

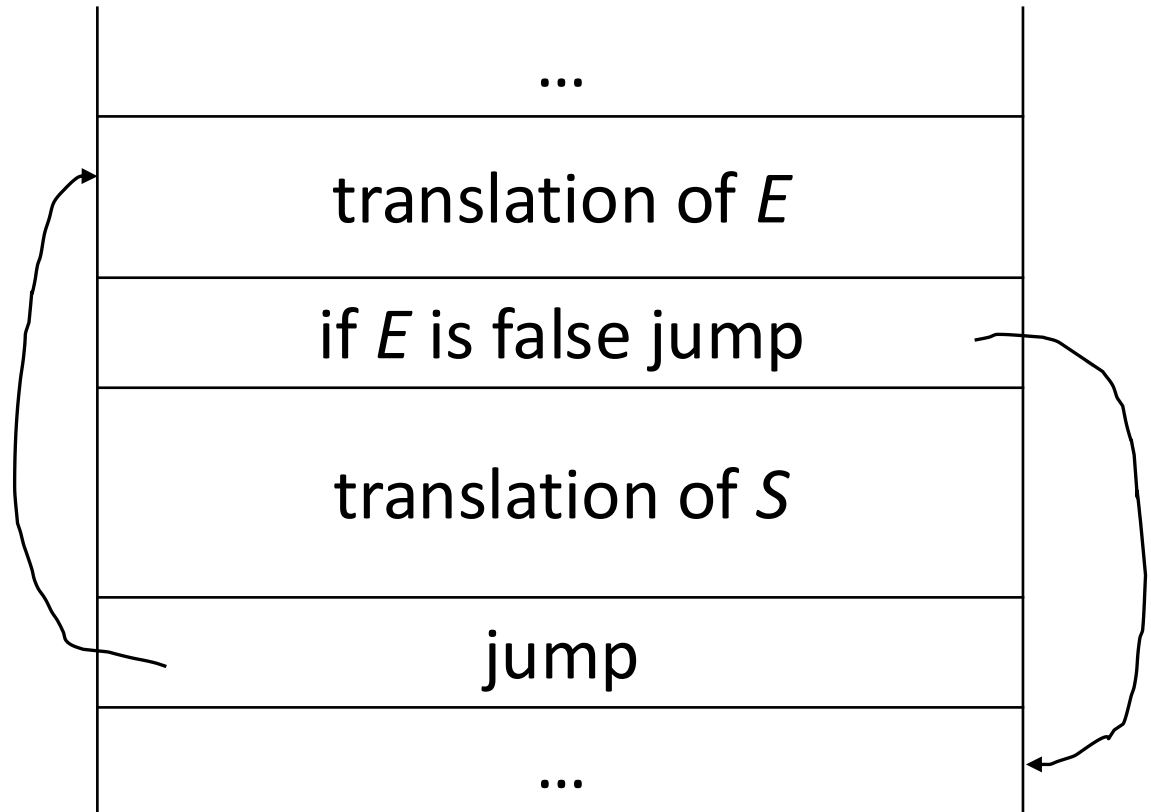
Hoare logic

Isao Sasano

Translation of statements

While statements are translated to efficient machine code.

Translation
of while
statements
of the form
while E do S



Translation of selection statements

Selection statements like case in Pascal can be translated in various ways, which may affect the programming style.

(ex. 1) A selection is translated to code with a jump table (array) , where a selection is recommended to be used only when the constants are almost adjacent.

(ex. 2) A selection is translated to code depending on the distribution of the constants. Selections with, say less than 7, cases are translated to nesting of conditionals.

Selection with a larger number of cases are translated to code with a jump table or hash table depending on the distribution of cases.

Syntax of statements in C

Statements in C are defined using extended BNF as follows.

```
<stmt> ::= ;  
          | <exp>;  
          | { <stmt-list> }  
          | if (<exp>) <stmt> | if (<exp>) <stmt> else <stmt>  
          | while ( <exp> ) <stmt>  
          | do <stmt> while (<exp>) ;  
          | for (<opt-exp>; <opt-exp>; <opt-exp>) <stmt>  
          | switch ( <exp> ) <stmt>  
          | case <const-exp> : <stmt>  
          | default : <stmt>  
          | break; | continue; | return; | return <exp>;  
          | goto <label>; | <label> : <stmt>  
  
<stmt-list> ::= <stmt> *  
<opt-exp> ::=  $\epsilon$  | <exp>
```

About C

C is different from Pascal in the following points.

- Basic types do not include the bool type. In conditional expression 0 is used as false and the others are used as true.
- = is the assignment operator, == and != are comparison operator.
- Conditional expressions in **while** statements must be surrounded by parentheses.
- Braces { and } are used in stead of **begin** and **end**.
- Logical **and** and **or** are && and ||.

Short circuit evaluation

Most languages like C, Modula2 and Pascal use *short-circuit evaluation* for the boolean **and** and **or** expressions. (In boolean expressions, the second operand is evaluated only when it is necessary.)

(ex.)

```
x = 1;
```

```
if (x == 1 || y == 2)
```

```
...
```

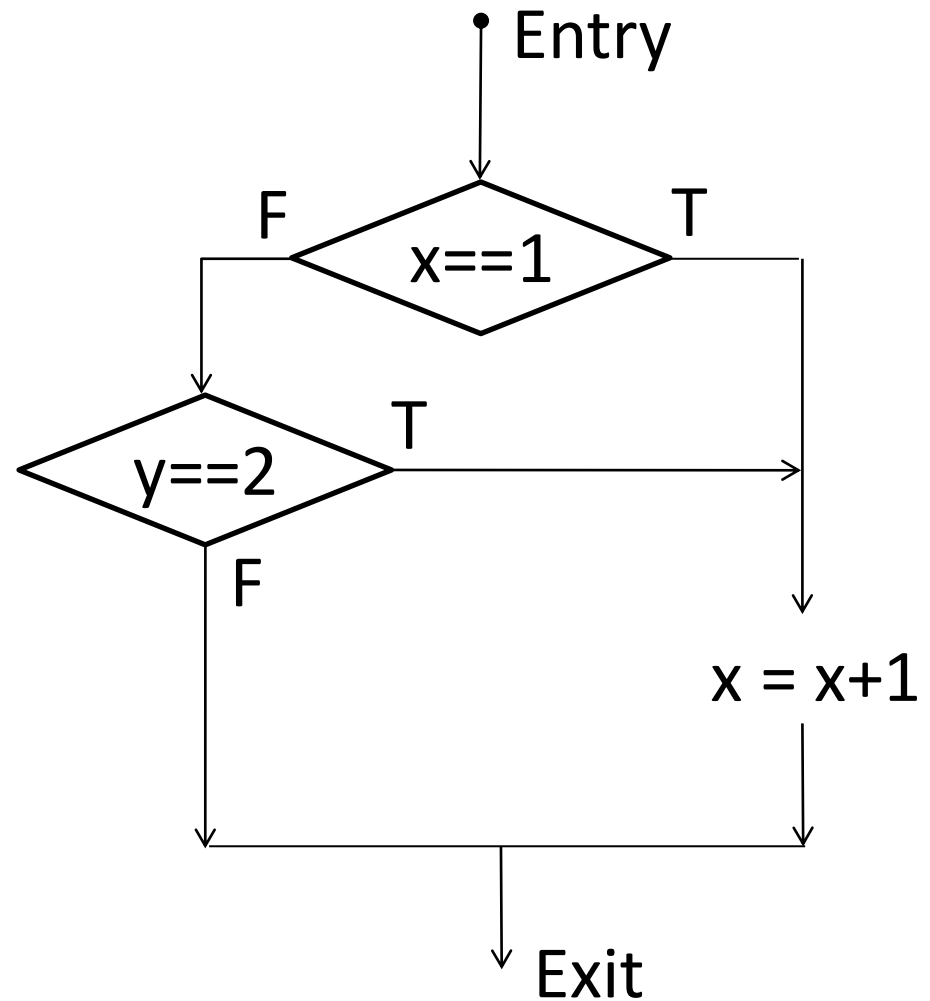
In the above code fragment in C, `x==1` is evaluated to 1 (true) so the conditional expression is evaluated to 1 (true) without evaluating the expression `y==2`.

Control flow in short-circuit evaluation

(ex.)

```
if (x==1 || y==2)  
    x = x + 1;
```

In short-circuit evaluation the control flow becomes a little complex but run-time efficiency is a little improved.



Tree-width (out of scope of the lecture)

Control flow graphs for programs without goto statements have complexity with respect to some criteria called *tree-width* no more than some constants.

- Algol without goto, Pascal without goto: ≤ 3
- Modula-2: ≤ 5 (Modula-2 does not have goto)
- C without goto: ≤ 6

The constants decrease by 1 without short-circuit evaluation. Different from Algol and Pascal, Modula-2 allows multiple exits (break) in a loop construct and multiple exits (return) in a function declaration. Additionally C has continue statements.

(Reference) Mikkel Thorup, “All structured programs have small tree-width and good register allocation”, *Information and Computation*, Vol. 142, pp. 159–181, 1998.

Exercise

(1) Illustrate the control flow graph of the following program fragment in C.

```
if (y==1 || y==2)
    if (x > 0)
        x = x - 1;
```

(2) Illustrate the control flow graph of the following program fragment in C.

```
while (x > 0) {
    if (x%2==0 || x%3==0)
        s = s + x;
    x = x - 1;
}
```

Assertions

An *assertion* is a conditional expression.

Java has a syntax for describing assertions.

In C++ we can use assertions as a macro by including `assert.h`.

(An ex. in Java)

```
int sum (int n) {  
    assert (n > 0);  
    ...  
}
```

If the programmer intends that the `sum` method always takes as its argument a positive integer, he can insert the assertion for finding bugs concerning this point.

Invariants

An *invariant* is a conditional expression (assertion) that holds every time control reaches a program point.

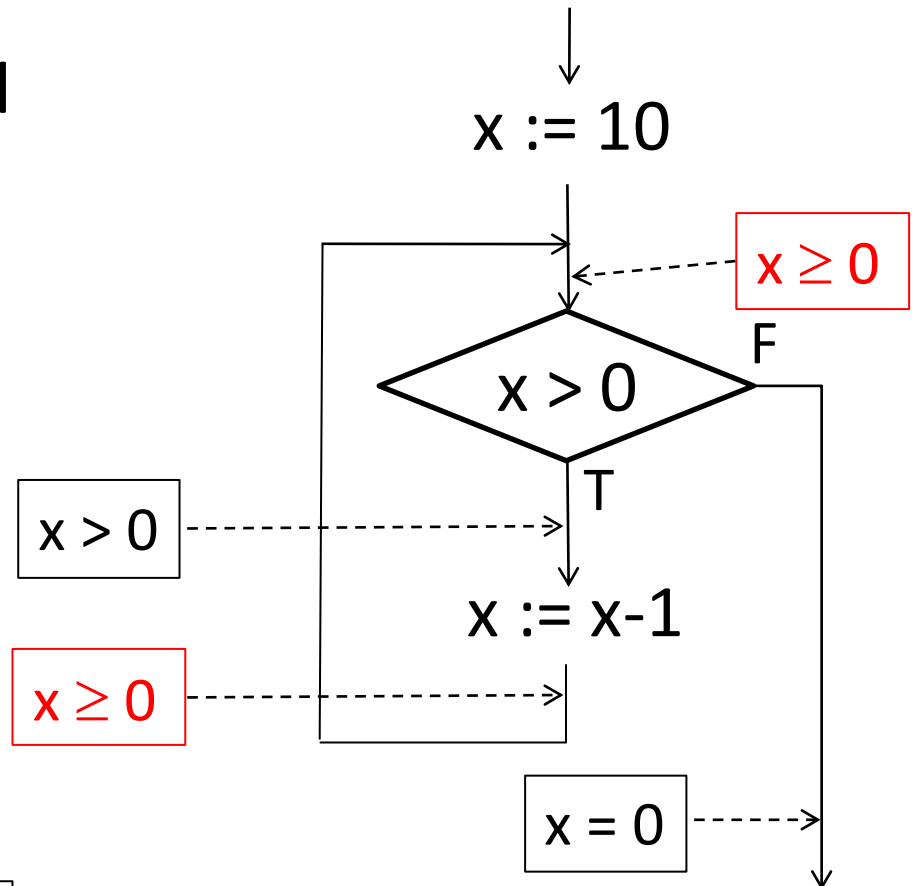
(ex.)

$x := 10;$

while $x > 0$ **do**

$x := x - 1$

We refer to the assertion
“ $x \geq 0$ ” as a loop invariant.



Values of variables are
assumed to be integers.

Examples of assertions

In this class we enclose assertions by curly braces.

(the previous ex.)

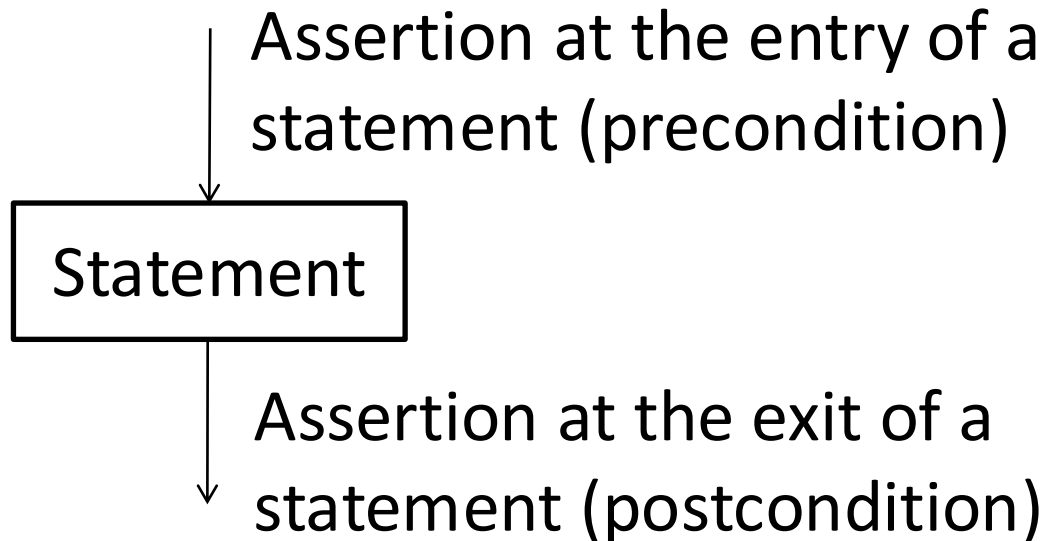
```
x := 10
{ x ≥ 0 }
while x > 0 do
  { x > 0 }
  x := x - 1
  { x ≥ 0 }
{ x = 0 }
```

The four assertions in the left program are all invariants.

Hoare logic is a logical system for proving this kind of things.

Precondition and postcondition

We can characterize the meaning of programs of single entry/single exit by assertions at the entry and the exit. (The meaning of a statement in an imperative language is the change of state before and after the statement.)



Hoare triple

We can describe the meaning of a statement by writing assertions before and after the statement. It was Charles Antony Richard Hoare (Tony Hoare in short) who proposed this notion. A statement with the assertions surrounded by curly braces $\{ \}$ is called a *Hoare triple*.

The meaning of
a Hoare triple
 $\{P\} S \{Q\}$

For any state σ that satisfies P , if the execution of S from the state σ terminates in σ' then σ' satisfies Q .

Examples of Hoare triple

$\{ a = 0 \} \ a := a + 1 \ \{ a = 1 \}$

For any state σ that satisfies $a=0$, if the execution of $a:=a+1$ from the state σ terminates in σ' then σ' satisfies $a=1$.

$\{ a = 1 \} \ a := a - 1; a := a + 1 \ \{ a = 1 \}$

For any state σ that satisfies $a=1$, if the execution of $a:=a-1; a:=a+1$ from the state σ terminates in σ' then σ' satisfies $a=1$.

$\{ a = 5 \} \ \mathbf{while} \ (a > 0) \ \mathbf{do} \ a := a - 1 \ \{ a = 0 \}$

For any state σ that satisfies $a=5$, if the execution of the while statement from the state σ terminates in σ' then σ' satisfies $a=0$.

Partial correctness

A Hoare triple $\{P\} S \{Q\}$ does not say that the statement S terminates (since a **while** statement may not terminate). In order to show the termination we need some other way. A Hoare triple is said to be a *partial correctness assertion*.

(ex.) $\{\text{true}\} \text{ while true do } x := 1 \{ \text{false} \}$

This Hoare triple holds.

(Note) Total correctness

--- Partial correctness + Termination

We may write $[P] S [Q]$ for a total correctness assertion.

(Note) By extending the while rule, we get a proof system for proving total correctness assertions.

Hoare logic

We present proof rules which generate valid Hoare triples. The proof rules are syntax-directed. *Hoare logic* is a proof system consisting of the collection of rules. Hoare logic is an axiomatic semantics. (There are various kinds of axiomatic semantics.)

(Note) Floyd considered similar thing on flowchart.

(Ref. 1) C. A. R. Hoare, "An axiomatic basis for computer programming", *Communications of the ACM*, 12(10):576–580,583, 1969.

(Ref. 2) R. W. Floyd, "Assigning meanings to programs", *Proceedings of the American Mathematical Society Symposium on Applied Mathematics*, Vol. 19, pp. 19–32. 1967.

Hoare logic

$$\frac{\{P\} S_1 \{Q\} \quad \{Q\} S_2 \{R\}}{\{P\} S_1; S_2 \{R\}} \quad (\text{composition rule})$$

$$\frac{\{P \wedge E\} S_1 \{Q\} \quad \{P \wedge \neg E\} S_2 \{Q\}}{\{P\} \text{ if } E \text{ then } S_1 \text{ else } S_2 \{Q\}} \quad (\text{conditional rule})$$

$$\frac{\{P \wedge E\} S \{P\}}{\{P\} \text{ while } E \text{ do } S \{P \wedge \neg E\}} \quad (\text{while rule})$$

$$\frac{}{\{Q[E/x]\} x := E \{Q\}} \quad (\text{assignment axiom})$$

$$\frac{P \Rightarrow P' \quad \{P'\} S \{Q'\} \quad Q' \Rightarrow Q}{\{P\} S \{Q\}} \quad (\text{consequence rule})$$

An example

We prove a Hoare triple $\{x \geq 0\}$ **while** $x > 0$ **do** $x := x - 1$ $\{x = 0\}$ by the rules of Hoare logic.

$$\begin{array}{c}
 \frac{}{\{x-1 \geq 0\} x := x - 1 \{x \geq 0\}} \text{ (assignment)} \\
 \frac{x \geq 0 \wedge x > 0 \Rightarrow x-1 \geq 0, \{x-1 \geq 0\} x := x - 1 \{x \geq 0\}}{\{x \geq 0 \wedge x > 0\} x := x - 1 \{x \geq 0\}} \text{ (consequence)} \\
 \frac{\{x \geq 0 \wedge x > 0\} x := x - 1 \{x \geq 0\}}{\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x \geq 0 \wedge \neg x > 0\}} \text{ (while)} \\
 \frac{\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x \geq 0 \wedge \neg x > 0\}, x \geq 0 \wedge \neg x > 0 \Rightarrow x = 0}{\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x = 0\}} \text{ (consequence)}
 \end{array}$$


(Note) There are more than one Hoare triple that holds for a statement.

(Note) We allow the consequence rule is applied to just one side.

Another example

	$r := x;$	
	$q := 0;$	$\{ \neg y \leq r$
$\{\text{true}\}$	$\text{while } y \leq r \text{ do}$	$\wedge$
	$(r := r - y; q := 1 + q)$	$x = r + y * q \}$

This Hoare triple states that for any state σ , if the execution of the statement from the state σ terminates in σ' , then σ' satisfies that the quotient and the remainder of x and y is stored at q and r respectively.

We omit the derivation (proof) of the above Hoare triple.

A note about assertions

As for assertions, in this class, we allow true, false, variables, integers, addition, subtraction, multiplication, comparison ($<$, $>$, $\leq$, $\geq$, $=$), logical operators ($\wedge$, $\vee$, $\neg$, $\Rightarrow$). Although $\forall$, $\exists$ can be used, in this class we do not use them. Although we should write rules for arithmetic operations, comparisons and so on, we do not write them and allow inferences for such kind to be done by common sense and be omitted.

(Ref. 1) Glynn Winskel, *"The formal semantics of programming languages"*, 1993, MIT Press, Chapter 6.

(Ref. 2) C. A. R. Hoare, "An axiomatic basis for computer programming", *Communications of the ACM*, 12(10):576–580,583, 1969.

Exercises

Derive (prove) the following Hoare triples.

(1) $\{ a = 0 \} \ a := a + 2 \ \{ a = 2 \}$

(2) $\{ a = 3 \} \textbf{if } a = 3 \textbf{ then } a := a + 1 \textbf{ else } a := a - 1 \{ a = 4 \}$

(3) $\{ a = 1 \} \textbf{while } a < 5 \textbf{ do } a := a + 1 \{ a = 5 \}$

A note (outside the scope of this class)

In actual computers basic types have limits and we need rules concerning the limits.

(ex.) overflow about the type of integers more than or equal to 0

(1) Program terminates with error if an overflow occurs.

$$\neg \exists x (x = \text{max} + 1)$$

(2) If an overflow occurs, we let the result to be the maximum number.

$$\text{max} + 1 = \text{max}$$

(3) If an overflow occurs, we let the result to be the remainder of the division by the maximum number.

$$\text{max} + 1 = 0$$